

第 2 章 C++简单程序设计

程序是由各种语句组成的，执行程序就是执行程序中的各条语句。语句规定了程序执行的控制流。本章主要介绍 C++ 的各种语句。学习本章要重点掌握各种语句的形式、功能及简单结构化程序设计的方法。通过本章的学习，能编写出简单的 C++ 程序。

2.1 程序的三种基本结构

程序流程是指程序中语句执行的顺序。程序流程有三种基本控制方式，即顺序控制、选择(分支)控制和循环控制。由这三种控制方式建立的程序就有三种基本结构，即顺序结构、选择结构和循环结构。

1. 顺序结构

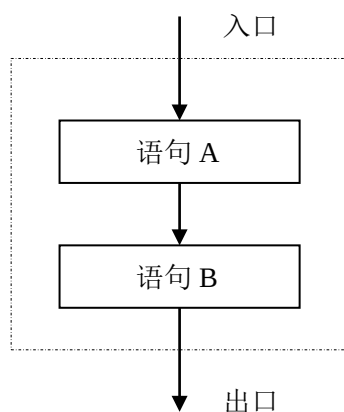


图 2.1 顺序结构

顺序结构是一种最简单、最基本的结构。这种结构的程序是按程序中语句出现的先后顺序执行的，上一章所举的例题都是顺序结构的程序。顺序结构的程序一般由表达式语句和输入输出流的使用等组成。

顺序结构程序见图 2.1。顺序结构的程序有一个入口、一个出口。执行时先执行语句 A，再执行语句 B，最后从出口出来继续执行下面的语句。

2. 选择结构(或称分支结构)

程序中有时需要有条件地选择执行某个语句。选择结构就是根据给定的条件决定执行哪个语句。选择结构可以分为单一条件选择和多条件选择。C++ 中实现选择结构的语句有 if 语句和 switch 语句。

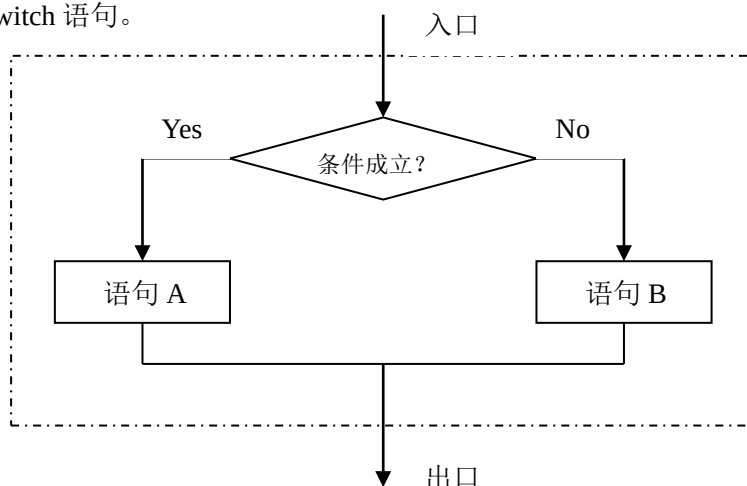


图 2.2 选择结构

选择结构示意图见图 2.2。进入该结构后，判断条件是否成立，成立则执行语句 A，否则执行语句 B。最后从出口出来继续执行下面的语句。其中的条件通常是关系表达式或逻辑表达式。

3.循环结构

程序中有时需要反复执行某个操作，实现这种操作的结构称为循环结构。循环语句就具有这个功能。它可以根据条件反复地执行某个语句。C++提供了三种循环语句，即 `while` 语句、`do-while` 语句和 `for` 语句。

循环结构有两种控制方式，见图 2.3。

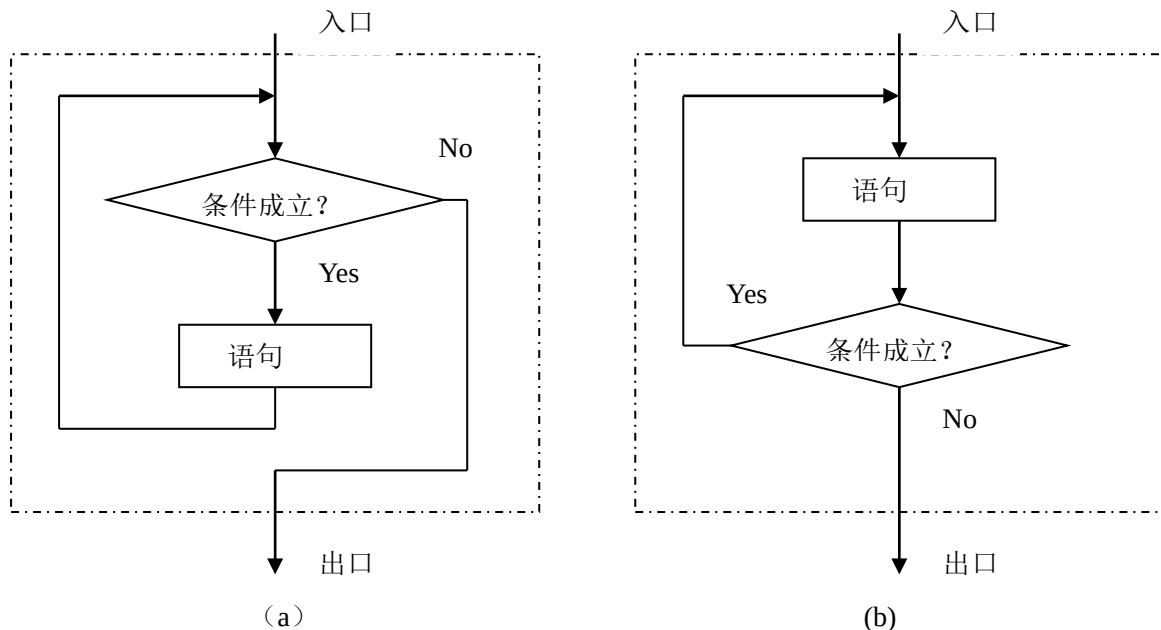


图 2.3 循环结构

(a)当型循环;(b)直到型循环

图 2.3(a)称为当型循环，即当条件成立时，反复执行语句部分，直到条件不成立，退出循环，继续执行循环结构后面的语句。图 2.3(b)称为直到型循环，先执行语句部分，再判断条件是否成立，若成立，再执行，反复进行，直到条件不成立，退出循环后继续执行后面的语句。

以上是结构化程序的三种基本结构。这三种结构的一个共同点是有一个出口和一个入口。无论是多么复杂的程序，结构化程序一般都是由这三种结构之间的简单衔接、复合或嵌套组成的。

从整体上看，一个选择语句或一个循环语句仍当做一个语句单元对待。因此从程序整体上看，构成函数的基本语句都是顺序执行的，即执行完一个语句后，紧接着执行下一条语句。用这种思想去编写和阅读程序，易于理解。

2.2 C++语句

在程序设计中，程序的功能是由语句实现的。C++语句可分为说明语句、表达式语句、控制语句、空语句和复合语句。

1.说明语句

说明语句包括函数原型说明、外部变量说明、结构类型说明、类的说明、模板说明等，例如：

```
int add(int,int);           //函数原型说明
```

```
extern double Radius; //外部变量说明
```

关于各种说明的形式和使用将在后面给出。

2.表达式语句

表达式语句是最简单也是用得最多的语句。在第 1 章介绍的每个表达式之后加上分号都可以构成一个表达式语句。例如：

```
i++;  
m+=5;  
x=1;  
y=(a>b?a+b:a-b);
```

都是表达式语句。只由一个分号构成的语句称为空语句。例如：

```
;
```

空语句在程序中不产生任何操作。

3.复合语句

在选择、循环等结构中又可包含有其他语句成分，形成一组语句序列，将这样一组语句序列用{}括起来就构成复合语句。复合语句又称“块语句”或“程序块”。复合语句在语法上等价于一个语句。下面是复合语句示例：

```
#include <iostream.h>  
void main()  
{ int a=5,b=6;  
  cout<<a<<b<<endl;  
  { int a=10;  
    cout<<a<<b<<endl;  
  }  
  cout<<a<<b<<endl;  
}
```

以上程序中的{int a=10;cout<<a<<b<<endl;}为复合语句。复合语句内可以有自己的变量。当复合语句内的变量与复合语句外的变量同名时，在复合语句内定义的变量有效。出了复合语句后，复合语句内定义的变量不起作用。

4.控制语句

控制语句包括选择语句、循环语句和转移语句。控制语句主要用来控制程序执行的顺序和过程，是本章的重点。控制语句除以上介绍的用于实现选择结构的 if 和 switch 语句和实现循环结构的 for、while 和 do-while 外，还包括用于实现程序控制转移的 break、continue、goto 和 return 语句。

5.函数调用和流对象调用语句

在函数调用之后加一个“;”就成为函数调用语句。例如：

```
strcpy(s1,s2);           //调用字符串拷贝函数  
cout<<a<<endl;          //标准输出流对象调用
```

调用的函数可以是库函数或用户定义的函数。

2.3 选择结构

程序一般是顺序执行的，即执行完一条语句后接着执行下一条语句。但是通过条件选择结构，可以改变程序的执行顺序，使一些语句被执行，而另一些语句不执行。实现选择结构

的语句有 if 语句和 switch 语句。

2.3.1 if 语句

if 语句有以下形式。

1.形式一

形式一的具体形式如下：

if(表达式)语句

if 语句中的表达式可以是任意类型表达式，语句可以是一个简单语句，也可以是几个语句组成的复合语句。形式一的执行流程见图 2.4(a)。

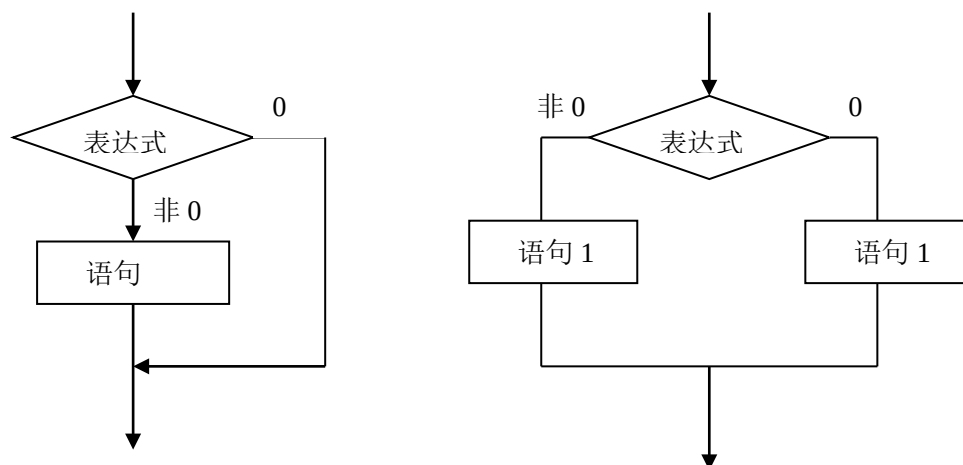


图 2.4 if 语句执行流程

(a)形式一 if 语句执行流程;(b)形式二 if 语句执行流程

形式一执行时先计算表达式的值。若表达式的值为非 0，执行语句部分;若表达式的值为 0 时，跳过语句部分，执行语句后边的其他语句。

例 2.1 输入一个数，若为正数，则原样输出;若为负数，求平方后输出。

```
#include <iostream>
using namespace std;
void main()
{ int a;
  cout<<" 请输入一个整数:" ; cin>>a;
  if(a<0) a*=a;
  cout<<" a=" <<a<<endl;
}
```

执行程序:

请输入一个整数:12
a=12

第二次执行程序:

请输入一个整数:-8
a=64

在例 2.1 中，根据输入的值判断 a。当 $a < 0$ 时，执行 $a*=a$ 语句后输出 a 的值;当 $a \geq 0$ 时，不执行 $a*=a$ 而直接执行后边的输出操作，输出 a 的值。

2.形式二

形式二的具体形式如下：

```

if(表达式)
    语句 1
else
    语句 2

```

形式二的执行流程见图 2.4(b)。形式二执行时先计算表达式的值。若表达式的值为非 0，执行语句 1 部分；若表达式的值为 0 时，执行语句 2 部分。无论执行语句 1 还是执行语句 2，执行后都执行后边的其他语句。

例 2.2 输入一个正整数。若该数为偶数，求 3 次方后输出；若该数为奇数，求平方值后输出。

```

#include <iostream.h>
void main()
{ int a,value;
  cout<<" 输入一个正整数:" ; cin>>a;
  if(a%2==0) value=a*a*a;
  else value=a*a;
  cout<<" value=" <<value<<endl;
}

```

执行程序:

```

输入一个正整数:5
value=25

```

第二次执行程序:

```

输入一个正整数:10
value=1000

```

当例 2.2 中输入一个数后，用 $a\%2==0$ 条件判断是否为偶数。若是，则 value 的值是 $a*a*a$ ；否则 value 的值是 $a*a$ 。最后输出 value 的值。

3.if 语句的嵌套

当 if 语句的某个语句部分中又出现另一个 if 语句时，称为 if 语句的嵌套。嵌套的 if 语句会出现多个 if 和 else，使用时一定要注意 else 与 if 的配对关系。应该是 else 总是与离它最近的且尚未匹配的 if 相匹配。例如：

```

if (a>=0)
    if(b>=0)
        cout<<a/b<<endl;
    else cout<<a*b<<endl;

```

中的 else 及 `cout<<a*b<<endl` 语句从格式上看好像是与 `if(a>=0)` 相匹配，但实际上 else 应与 `if(b>=0)` 相匹配。若要想使 else 与 `if(a>=0)` 匹配，应写成以下形式：

```

if(a>=0)
{
    if(b>=0)
        cout<<a/b<<endl;
    else cout<<a*b<<endl;
}

```

由于 else 不在 `if(b>=0)` 所在的复合语句中，因此复合语句

```

{ if (b>=0) cout<<a/b<<endl;}

```

作为条件 $(a>=0)$ 满足时的执行部分，这样写就不会引起错误，而且增加了程序的可读性。因此在这种结构的程序设计中常常有意识地加一些 { }，把嵌套的部分括起来。

例 2.3 输入一个成绩，给出该成绩的等级。

成绩 ≥ 90	优
$80 \leq \text{成绩} < 90$	良
$70 \leq \text{成绩} < 80$	中
$60 \leq \text{成绩} < 70$	及格
成绩 < 60	不及格

程序如下：

```
#include <iostream>
using namespace std;
int main( )
{ float score;
  cout<<" 输入成绩(0~100):" ; cin>>score;
  if(score<0 || score>100)
    cout<<" 输入错!\n" ;
  else if (score>=90)
    cout<<" 优\n" ;
  else if (score >=80)
    cout<<" 良\n" ;
  else if (score >=70)
    cout<<" 中\n" ;
  else if (score >=60)
    cout<<" 及格\n" ;
  else  cout<<" 不及格\n" ;
  return 0;
}
```

执行程序：

输入成绩(0~100):85

良

第二次执行：

输入成绩(0~100):68

及格

第三次执行：

输入成绩(0~100):54

不及格

这种嵌套的 if 语句用来表示多分支结构，其中的条件应该是互不相同的，任何时候只有一个条件满足。

例 2.4 求三个数中的最大数。

```
#include <iostream.h>
int main()
{ float a,b,c,amax;
  cout<<" 请输入 3 个数:" ; cin>>a>>b>>c;
  amax=a;
  if(b>c)
  { if(b>a) amax=b;}
```

```

else
    if(c>a) amax=c;
cout<<" max=" <<amax<<endl;
return 0;
}

```

该程序也可以写成以下形式:

```

#include <iostream.h>
int main()
{ float a,b,c,amax;
  cout<<" 请输入 3 个数:" ; cin>>a>>b>>c;
  if(a>=b)
  { if(a>=c) amax=a;
    else amax=c; }
  else { if(b>=c) amax=b;
        else amax=c; }
  cout<<" max=" <<amax<<endl;
  return 0;
}

```

使用 if 语句时要注意以下几点:

①if 中的表达式可以是任意形式,只要结果是 0 或非 0 值即可。因此 if 中的表达式可以有各种写法,例如:

```

if(5)
if(' a' )
if(!x)
if(m=x)
if(m==x)
if(a>b?a:b)
if(a++)
if(x>10 && x<20)

```

等等都是正确的表达式形式。初学者往往把 if(m==x)误写为 if(m=x),前者是判断 m 是否等于 x;而后者则是将 x 的值赋予 m 后,判断 m 的值是 0 或非 0。这种错误编译系统不会发现,因为它们都是合法的 C++语句。

②若 if 或 else 后的执行部分为两条以上的语句时,一定要写成复合语句形式,否则仅执行 if 后的第一条语句。这种逻辑上的错误,编译程序是检查不出来的。

2.3.2 switch 语句

switch 语句是多分支选择语句,基本形式为:

```

switch(表达式)
{ case 常量值 1:语句 1[break;]
  case 常量值 2:语句 2[break;]
  :
  case 常量值 n:语句 n[break;]
  [default:语句 n+1 [break;]]
}

```

说明如下:

①switch (表达式)中的表达式可以是整型、字符型或枚举型表达式;

②每个 case 后的“常量值 i”必须是常量, 并且每个常量值都不能相同, 它们的类型应与 switch(表达式)中表达式的类型相同;

③每个 case 和 default 的次序可以任意, 不影响执行结果。

switch 语句的执行过程是:先计算表达式的值, 用这个表达式值与每个 case 后的常量值进行比较, 与哪个常量值相等, 就执行相应常量值后的语句, 直至遇到 break 语句, 结束 switch 语句。如果没有相等的常量值, 而有 default 语句, 则执行 default 后的语句;如果没有 default 语句, switch 语句将不执行任何语句, 直接执行 switch 后的语句。switch 语句执行流程见图 2.5。

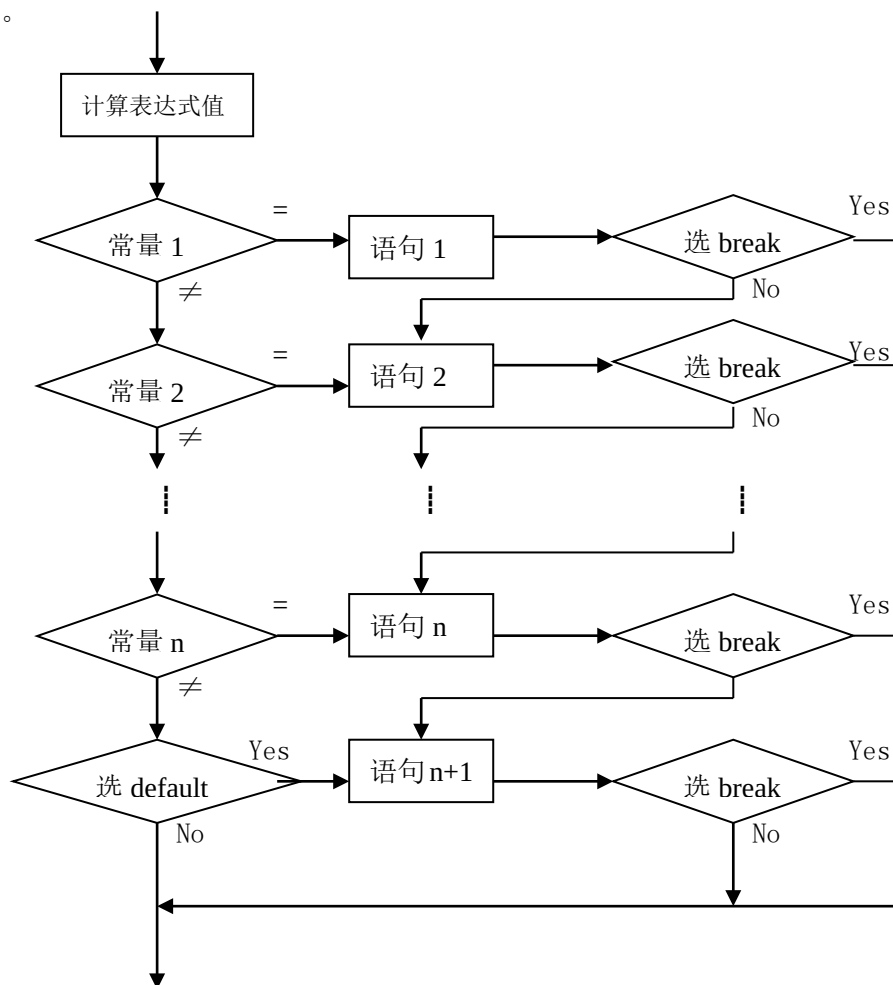


图 2.5 switch 语句执行流程

在 switch 语句中, default 和每个语句后的 break 语句都为可选项。若没有选择 break 语句, 执行完与常量值相匹配的语句后, 不退出 switch 语句, 而继续执行下一个 case 后的语句, 直至遇到 break 语句或 switch 最后的右花括号。每个常量值后的语句 i 可以是复合语句。无论 default 出现在 switch 语句中的哪一个位置上, 都表示所有其他条件不满足时所执行的操作。

例 2.5 分析以下程序运行结果。

```
#include <iostream.h>
void main()
```

```

{ int x,a,b,c,d;
  a=b=c=d=0;
  cin>>x;
  switch(x)
  {   case 1: a++; break;
      default: d=1;
      case 2: b++; break;
      case 3: c++; break;
  }
  cout<<a<<b<<c<<d<<endl;
}

```

执行程序:

当输入 1 时, 执行结果为 1000

当输入 2 时, 执行结果为 0100

当输入 5 时, 执行结果为 0101

将例 2.5 程序中的 switch 语句改写成如下形式:

```

#include <iostream.h>
void main()
{ int x,a,b,c,d;
  a=b=c=d=0;
  cin>>x;
  switch (x)
  {   case 1: a++;
      case 2: b++;
      case 3: c++; break;
      default: d=1; break;
      case 0 : ;           //空语句
  }
  cout<<a<<b<<c<<d<<endl;
}

```

执行程序:

当输入 1 时, 执行结果为 1110

当输入 2 时, 执行结果为 0110

当输入 5 时, 执行结果为 0001

当输入 0 时, 执行结果为 0000

2.4 循环结构

循环是所有程序设计语言最基本的功能。通过循环可以反复执行某些语句, 可以设置反复执行的过程、反复执行的条件或反复执行的次数等。反复执行的过程称为循环体。在编写循环结构程序时, 一般都要考虑两个问题:

- ①有一个控制循环的变量或称计数器, 它有初始值, 并在循环执行过程中不断地被修改;
- ②确定一个条件, 当该条件成立(为真)时, 循环被执行, 否则循环结束。

C++提供了三种实现循环的语句, 即 while、do-while 和 for 语句。

2.4.1 while 语句

while 语句的形式为:

```
while(表达式)
    循环体
```

其中表达式可以为各种类型表达式,是决定循环体是否被执行的条件。循环体可以是一个语句,也可以是复合语句。

while 语句执行过程是:首先计算表达式的值,若表达式值为非 0(为真),则执行循环体内各语句,然后再去计算判断表达式值。重复上述过程,直至表达式的值为 0(为假)时,结束循环,转去执行循环体后面的语句。它属于当型循环,即当条件成立时执行循环。while 语句执行流程见图 2.3(a)。

由于 while 语句是先判断,再执行,所以如果第一次判断表达式的值就不成立,循环体可能一次也不执行。

例 2.6 求自然数 1~100 的累加和。

```
#include <iostream.h>
void main()
{ int i(0), sum(0);
  while(i<101)
  { sum+=i; i++; }           //循环体
  cout<<" sum=" <<sum<<endl;
}
```

执行结果为:

sum=5050

在例 2.6 中变量 i 控制循环是否执行,初始值为 0,当 i<101 时,执行一遍循环体。然后再判断 i<101 是否成立。i 的值在循环体内每次加 1,直到 i 等于 101 时,不满足条件,结束循环,程序执行循环体后边的 cout 语句。

该程序也可以写成以下形式:

```
#include <iostream.h>
void main()
{ int i(0),sum(0);
  while(i++<100)
  { sum+=i;           ?//循环体
    cout<<" sum=" <<sum<<endl;
  }
```

注意这里循环结束的条件 (i++<100)。执行时先判断 i 的值是否小于 100,判断后在执行循环体之前,i 加 1,第一次循环体执行的是 sum+1;第二次先用 i 中的 1 判断是否小于 100,条件成立,i 加 1,第二次循环体执行的是 sum+2。如此进行,当 i 用 99 判断是否小于 100 时,循环体执行的是 sum+100。最后 i 是 100 条件不满足,结束循环。

此程序修改 i 的初始值,循环语句可以写成以下形式:

```
int i(1),sum(1);
while(i++<100)
{ sum+=i;           //循环体
```

或者

```
int i(0),sum(0);
```

```
while(++i <=100)
    sum+=i;    //循环体
```

2.4.2 do-while 语句

do-while 语句形式为:

```
do 循环体
while(表达式);
```

其中表达式和循环体的含义与 while 循环相同。与 while 循环不同的是，它先执行循环体，然后再判断条件是否满足，若满足，则再次执行循环体。如此重复进行，直至表达式的值为 0 时结束循环。它属于直到型循环，执行流程见图 2.3(b)。和 while 循环相比，直到型循环的循环体至少执行一次。

注意在 do-while(表达式)后有一分号“;”，作为整个 do-while 语句的结束。

例 2.7 输出 A~Z 英文大写字母。

```
#include <iostream.h>
void main()
{ char ch;
  ch=' A' ;
  do{ cout<<ch;
      ch++;          //字符的 ASCII 码值加 1
  }while(ch<=' Z' );
}
```

用 while 循环编写的程序也可以用 do-while 循环实现。将例 2.6 用 do-while 完成，即

```
#include <iostream.h>
void main()
{ int i(0),sum(0);
  do { sum+=i++; }while(i<=100);
  cout<<" sum=" <<sum<<endl;
}
```

为避免无限循环，在 while 和 do-while 循环体内(或测试条件内)应存在使循环趋于结束的语句。例如:

```
i=5;
do { cout<<i<<endl;
      i=i-2;
  }while(i!=0);
```

中的 i 使表达式(i!=0)条件总成立，循环体就会无限次执行。又如:

```
int ok;
while (ok=1)
{ cout<< "输入 ok 值:" ;
  cin>>ok;
}
```

执行时程序会无限次地提示“输入 ok 值:”，原因是将 while (条件)中的“条件”写错了。初学者编写循环程序时注意避免这类错误。

2.4.3 for 语句

for 循环语句的形式为:

for(e1 ;e2 ;e3)

循环体

其中 e1、e2、e3 为表达式。一般 e1 为赋值表达式，用于为循环中的变量赋初始值;e2 一般是关系或逻辑表达式，作为继续循环的条件;e3 通常为增、减量运算，用于修改循环条件中的变量。e1、e2 和 e3 之间用 “;” 号分隔。

for 语句中的 e1、e2 和 e3 任何一个表达式都可以省略，但 “;” 不能省略。

for 语句中循环体可以是复合语句。

for 循环的执行过程是:首先计算 e1 的值，然后判断 e2 的值。若 e2 非 0，则执行一遍循环体后，进行 e3 表达式的计算，计算后再去判断 e2 表达式;若 e2 非 0，再执行一遍循环体。如此重复进行，直到 e2 表达式的值为 0 时，结束循环。

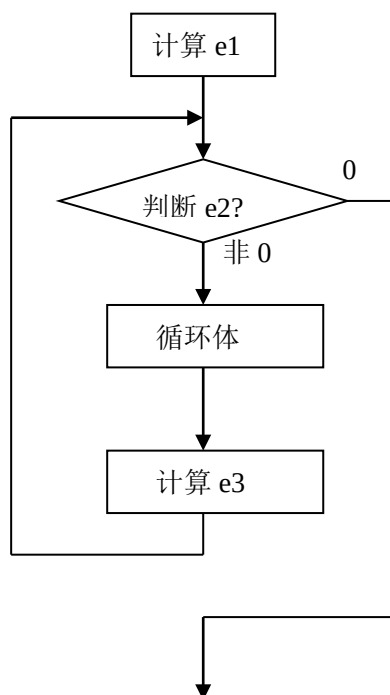


图 2.6 for 语句执行流程

for 语句的执行流程见图 2.6。

例 2.8 输出 1 到 10 各数的平方值。

```
#include <iostream.h>
void main()
{ int i;
  for(i=1; i<=10; i++)
    cout<<i<<" " <<i*i<<endl;
}
```

for 循环语句中的 e1 表达式只计算一次，可以放在 for 循环的前面。e3 表达式每次执行完循环体后都要计算一次，可以放在循环体内。例 2.8 程序可以写成以下形式:

```
#include <iostream.h>
void main()
{ int i=1;
  for (; i<=10; )
  { cout<<i<<" " <<i*i<<endl;
```

```

        i++;
    }
}

```

该程序的 for 语句中省略 e1 和 e3 表达式。如果三个表达式都省略，for 语句的形式为：

```

for ( ;; )
    循环体

```

这时在循环体内应该有控制循环执行和结束的语句，否则会出现死循环。

for 语句中的 e1、e2 和 e3 也可以使用逗号表达式，见例 2.9 题。

例 2.9 求 10!

```

#include <iostream.h>
void main()
{ for(int i=1, long p=1; i<=10; i++) p*=i;
  cout<<" 10!=" <<p<<endl;
}

```

运行结果为：

```

10!=3628800

```

2.4.4 多重循环

多重循环又称循环的嵌套，即在一个循环语句的循环体内又包括另一个循环语句。上述介绍的三种循环语句在 C++ 中不仅可以自身嵌套，还可以互相嵌套。嵌套时应注意必须将一个完整的循环结构全部放在另一个循环体内。

例 2.10 分析以下程序运算结果。

```

#include <iostream.h>
#include <iomanip.h>
void main()
{ int i,j;
  for(i=1; i<10; i++)
  { j=0;
    while (++j<=10-i)
      cout<<setw (4) <<i*j;
    cout<<endl;
  }
}

```

运行结果为：

```

1  2  3  4  5  6  7  8  9
2  4  6  8 10 12 14 16
3  6  9 12 15 18 21
4  8 12 16 20 24
5 10 15 20 25
6 12 18 24
7 14 21
8 16
9

```

对例 2.10 程序说明如下：

①这是个二重循环的程序，程序外层 for 循环的循环体内又包括一个 while 循环语句。

②程序执行时，for 循环语句循环体每执行一次，内层的 while 循环由开始执行直到 while 循环结束，退出 while 循环接着再执行外层循环的其他语句。当外层循环条件满足时，再由外层循环进入内层循环，内层 while 循环再次从头到尾执行。如此进行，直到外层 for 循环终止时，循环结束。

③程序内层 while 循环语句的循环体只有一条语句：`cout<<setw(4)<<i*j<<" "`。每当结束 while 循环后，接着执行 `cout<<endl` 语句实现换行。

④该程序外层 for 循环语句控制行数，内层 while 循环语句控制每行输出的内容。在设计多重循环时，应注意内层循环控制循环次数的变量不要与外层循环控制变量重名。例如：

```
for(i=1; i<10; i++)
```

```
for(i=0; i<5; i++)
```

是错误的，而编译程序不检查此类错误。

2.5 跳转语句

跳转语句是辅助性语句，不单独使用，一般与选择语句或循环语句结合起来使用，起到控制程序流程的作用。跳转语句包括 break、continue 和 goto 语句。

2.5.1 break 语句

break 语句格式为：

```
break;
```

break 语句有两个用途：一是用于 switch 语句；二是用于循环语句。当在循环体执行过程中遇到 break 语句时，终止循环的执行，中途退出循环，转去执行循环体后面的语句。

例 2.11 判断输入的某正整数是否为素数。素数是除 1 和本身外不能被其他任何数整除的数。

```
#include <iostream.h>
void main()
{ int m;
  do{ cout<<" 输入一个正整数:" ;
      cin>>m;
    }while(m<=0);
  for (int i=2; i<m; i++)
    if (m%i==0) break;
  if (i==m) cout<<m<<" 是一个素数\n" ;
  else cout<<m<<" 不是一个素数\n" ;
}
```

第一次执行程序：

输入一个正整数:431

431 是一个素数

第二次执行程序：

输入一个正整数:69

69 不是一个素数

这个程序中的 do-while 循环的作用是确保输入的 m 是一个正整数后，才去判断其是否为素数。在 for 循环中，若 m 能被 2~m-1 中的任一个数 i 整除，说明不是素数，则执行 break 语句退出该循环。for 循环正常结束后，条件(i==m)成立，表示 2~m-1 中的任一数均不能整

除 m ，故 m 是素数。

在多重循环中使用 `break` 语句时，只能退出 `break` 语句所在层的循环，不能退出上一层的循环。

2.5.2 continue 语句

`continue` 语句格式为:

```
continue;
```

`continue` 语句只用于循环语句中。当在循环体内遇到 `continue` 语句时，它中断本次循环体的执行，立即执行下一次循环。

`continue` 语句与 `break` 语句不同。`break` 是终止执行循环语句，退出循环;而 `continue` 是中断本次循环，不再执行循环体内 `continue` 语句后面的语句，而继续下一次循环。

例 2.12 将 100 以内能被 3 整除的数输出，每行输出 10 个数。

```
#include <iostream.h>
void main()
{ int k=0;
  for(int i=1; i<100; i++)
  { if ( i%3 !=0) continue;
    cout<<i<<" ";
    if(++k%10==0) cout<<endl;    //控制每行输出 10 个数
  }
  cout<<endl;
}
```

2.5.3 goto 语句

`goto` 语句格式为:

```
goto 标号;
```

其中标号在 C++ 中与变量名命名相同。使用时放在语句的前边，后跟一个冒号。例如:

```
loop:语句
```

其中 `loop` 为一标号。

`goto` 语句的作用是无条件地转去执行标号所指出的语句。

C++ 中 `goto` 语句的使用被限制在一个函数内。在同一个函数内，语句标号是唯一的。使用 `goto` 语句会破坏程序的结构，使程序可读性差，因此在编程时最好不用。只有在某些情况下使用 `goto` 语句会起到提高程序效率的作用。在多重循环的结构中，可以用 `goto` 语句由最内层循环直接跳到最外层循环，或跳到所有循环层之外。这一点其他语句是做不到的。

例 2.13 求 100 以内偶数的和。

```
#include <iostream.h>
int main()
{ int s=2,i;
  for(i=3; i<=100;)
  { if( i%2 ) goto kk;
    s+=i;
    kk: i++;
  }
  cout<<" sum=" <<s<<endl;
```

```

        return 1;
    }
    执行结果为:
    sum=2550

```

2.6 例题

例 2.14 分析以下 switch 语句执行后的输出结果。

```

#include <iostream>
using namespace std;
void main()
{ int a=1,b=2,c=0,d=0;
  switch (a)
  { case 1: switch(b)
    {   case 1: c++; break;
        case 2: d++; break;
    }
    case 2: c++; d++; break;
  }
  cout<<" c=" <<c<<endl;
  cout<<" d=" <<d<<endl;
}

```

执行结果为:

```

c=1
d=2

```

该程序的 switch (a)中, 当 case 常量值为 1 时, 语句部分又出现了一个 switch 语句。阅读该程序, 注意 break 语句的作用。

例 2.15 利用公式 $e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \cdots + \frac{1}{n!} + \cdots$ 求 e 的近似值, 直到 $\frac{1}{n!} \leq 10^{-7}$ 为止。

输出 n 及 e 的值。

程序如下:

```

#include <iostream>
using namespace std;
int main()
{ int n;
  double e,u;
  n=1; e=u=1.0;
  do{ u/=n;
    e+=u;
    n++;
  }while(u > 1.0e-7);
  cout<<" n=" <<n<<" " <<" e=" <<e<<endl;
  return 0;
}

```

程序执行结果:

n=12 e=2.71828

该程序预先不知道循环次数, 而第 1 次又必须执行时, 所以采用 do-while 循环。程序中语句 $u/=n$ 第一次求的是 $\frac{1}{1!}$, 第二次循环时用 $\frac{1}{1!}$ 除以 2, 得到 $\frac{1}{2!}$, 如此循环, 直到满足要求。

例 2.16 打印大写 26 个英文字母, 每行打印 6 个字母.

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{ char letter;
  int count(0);
  cout<<"    大 写 英 文 字 母 表\n" ;
  for (letter=' A' ; letter<=' Z' ; letter++)
  { cout<<setw(5)<<letter;           //每个打印的字母占 5 位宽度
    count++;                         //计数器加 1
    if (count==6)                   //打印够 6 个字母时, 输出新行
    { cout<<endl; count=0; }
  }
  return 0;
}
```

例 2.17 编程输出以下图案:

```

      1
    2  2
  3  3  3
4  4  4  4
5  5  5  5  5
6  6  6  6  6  6
7  7  7  7  7  7  7
8  8  8  8  8  8  8  8
9  9  9  9  9  9  9  9  9
```

程序如下:

```
#include <iostream >
using namespace std;
void main()
{ for (int i=1; i<10; i++)
  { for(int j=1; j<=12-i; j++)      //输出每行前的空格
    cout<<" ";
    for (j=1; j<=i; j++)           //输出各行值
      cout<<j<<" ";
    cout<<endl;                   //在新行输出
  }
}
```

练习 2

1.回答以下问题:

- (1)结构化程序有哪几种基本结构?
- (2)什么是复合语句?什么是空语句?
- (3)if(e)中的 e 可以是什么类型的内容?
- (4)在循环语句中使用 break 和 continue 语句时有什么区别?
- (5)switch 语句中的 break 作用是什么?
- (6)哪种循环的循环体至少执行一次?
- (7)检查一个浮点数是否为 0 的最好方法是什么?

2.阅读下列程序并写出运行结果:

(1) #include <iostream>

```
using namespace std;
```

```
void main()
```

```
{ int a=2,b= -3,c=3;
```

```
if(a=b+c) cout<<" ####" <<endl;
```

```
else cout<<" ****" <<endl;
```

```
}
```

(2) #include <iostream >

```
using namespace std;
```

```
void main()
```

```
{ int a=5;
```

```
if (a++>5) cout<<a<<endl;
```

```
else cout<<a--<<endl;
```

```
}
```

(3) #include <iostream>

```
using namespace std;
```

```
void main()
```

```
{ for (int k=1, int s=0 ; k<=10; k++);
```

```
s+=k;
```

```
cout<<" s=" <<s<<endl;
```

```
}
```

(4) #include <iostream >

```
using namespace std;
```

```
void main()
```

```
{ int i=0,s=0;
```

```
for(;;)
```

```
{ i+=2;
```

```
if (i>8) { cout<<s<<endl; break;}
```

```
if (i==6) continue;
```

```
s+=i;
```

```
}
```

```
}
```

(5) #include <iostream >

```

using namespace std;
void main()
{   int n, sum;
    do{ cin>>n;   sum=0;
        for (int i=0; i<5; i++)
        {   sum+=n%10;   n=n/10;
            if((i==4)||(n==0)) break;
        }
        cout<<" sum=" <<sum<<endl;
        if(sum==0) break;
    }while (1);
}

```

执行该程序，当分别输入 5238、316、24、0 时，输出结果是什么？

3.用 if 语句编写程序，计算下式：

$$y = \begin{cases} n+1 & n>0 \\ 0 & n=0 \\ n^2 & n<0 \end{cases}$$

4.编写程序，输入 10 个学生的考试分数(0~100)，然后根据每个学生的分数，判断并显示每个学生的学号和等级。学号由 1 到 10，等级关系为：

$$\text{等级} = \begin{cases} \text{A} & 90 \leq \text{分数} \leq 100 \\ \text{B} & 80 \leq \text{分数} < 90 \\ \text{C} & 60 \leq \text{分数} < 80 \\ \text{D} & 0 \leq \text{分数} < 60 \end{cases}$$

当分数<0 或分数>100 时，显示输入错并重新输入。

5.编写程序，提示用户输入一个字符，判断输入的字符是数字(0~9)、小写字母(a~z)、大写字母(A~Z)或者其他字符。

6.编写程序，用公式 $\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots$ 求 π 的近似值，直到最后一项的绝对值小于 10^{-6} 为止。

7.斐波那契数列中的前两个数是 0 和 1，从第三个数开始，每个数等于前两个数之和，即 0、1、1、2、3、5、8、13……。编程序求斐波那契数列的前 40 个数，每行输出 8 个数，分 5 行输出。

8.用 switch 语句编写程序，统计输入的一串字母中元音字母(a、e、i、o、u)的总个数和每个元音字母出现的次数，当输入“#”时，输入结束。

9.摄氏温度(C)和华氏温度(F)的转换公式为：

$$C = \frac{5}{9} \times (F - 32)$$

编程序输出华氏-30 度到 100 度并间隔为 5 度的华氏温度和摄氏温度的对照表。

10.编程序计算 $5!+6!+7!$ 。

11.编程序求从 2 开始连续 100 个素数。

12.百钱百鸡问题是我国古代数学家一道解不定方程的问题。设公鸡每只 5 元，母鸡每只 3 元，小鸡每 3 只 1 元。用 100 元钱买 100 只鸡，问公鸡、母鸡和小鸡各买多少只？编程实现。

13.编写程序验证：任何一个自然数 n 的立方都可以写成 m 个连续的奇数之和。例如：

$$1^3=1$$

$$2^3=3+5=8$$

$$3^3=7+9+11=27$$

$$4^3=13+15+17+19=64$$

14.编写程序，显示输出如下的图形:

```

      *
    * * *
  * * * * *
* * * * * * *
  * * * * *
    * * * *
      * * *
        *
```